Ders Adı	YMÜ227 Nesne Tabanlı Programlama
Proje Adı	Sayı Tahmin Oyunu (Console Application)
Dersin Hocası	Sinem AKYOL
Öğrenci Bilgileri	240290040 Abdullah Batur DEMİR
Proje Tarihi	26 Ekim 2025

İÇİNDEKİLER

1. Giriş – Proje Amacı

2. Geliştirme Süreci 2.1. Sınıf Tasarım Gerekçeleri 2.2. Oyun Akışı 2.3. Karşılaşılan Sorunlar ve Çözümleri

3. Kaynak Kodlar 3.1. Çalıştırma Adımları 3.2. Kod Listesi (Yorumlu)

4. Çıktılar / Sonuç ve Değerlendirme 4.1. Test Senaryoları 4.2. Oyun Oturumu Örnekleri (Çıktılar)

1. Giriş – Proje Amacı

Bu proje, YMÜ227 Nesne Tabanlı Programlama dersi kapsamında, Java dili kullanılarak geliştirilen konsol tabanlı bir Sayı Tahmin Oyunu uygulamasıdır. Projenin temel amacı, belirlenen bir aralıkta rasgele üretilen gizli bir sayıyı, kullanıcıya tanınan kısıtlı deneme hakkı içinde buldurmaktır.

Proje Amaçları:

- Nesne Tabanlı Programlama (OOP) temel prensiplerini (Kapsülleme, Sorumlulukların Ayrılması) uygulamak.
- Uygulama akışı ve kullanıcı girdileri için sağlam bir hata yönetimi mekanizması kurmak.
- Uygulamanın mantıksal ve yapısal bileşenlerini farklı sınıflara ayırarak modüler bir mimari oluşturmaktır.

2. Geliştirme Süreci

2.1. Sınıf Tasarım Gerekçeleri

Proje, her bir bileşenin tek bir sorumluluğu üstlenmesi (Single Responsibility Principle) esasına göre tasarlanmıştır.

Sınıf Adı	Temel Sorumluluk Alanı	Tasarım Gerekçesi
`App.java`	Kullanıcı Arayüzü (Konsol I/O), Uygulama Akışı, Yapılandırma.	Kullanıcı etkileşimini, oyun mantığından ayırır. Kontrolcü (Controller) rolündedir.
`NumberGuessGame.java`	Oyun Durumu Yönetimi, Tahmin Değerlendirme Mantığı.	Oyunun temel durumunu ve kurallarını kapsüller. Model rolündedir.
`GameConfig.java`	Oyun Ayarlarını (min, max, hak) sabit tutma.	Konfigürasyon verilerinin güvenli ve tek bir noktadan yönetilmesini sağlar.
`RandomNumberGenerator.java`	Belirlenen aralıkta rasgele sayı üretme işini soyutlama.	Rasgele sayı üretim metodunu diğer sınıfların karmaşasından ayırır.
`Attempt.java`	Bir denemenin sonucunu (`GuessResult`) ve detaylarını (numarasını) temsil etme.	Sonuç enum'ını mantıksal olarak ilişkili olduğu bu sınıfa yerleştirir.

2.2. Oyun Akışı

1. **Başlatma ve Güvenlik:** `App.java`'daki `main` metodu çalışır. `java.util.Scanner` nesnesi, kaynak sızıntısını önlemek için `try-with-resources` yapısı içinde oluşturulur.
2. **Yapılandırma:** `configureGame` metodu, kullanıcıdan oyunun aralığını (1-100 veya 1-1000) ve deneme hakkını ister. Aralık seçimi, sadece **'1' veya '2'** girilene kadar döngüsel olarak doğrulanır.
3. **Oyun Kurulumu:** `GameConfig` oluşturulur ve `NumberGuessGame` başlatılır. Gizli sayı `RandomNumberGenerator` ile üretilir.
4. **Ana Döngü:** `while (!game.isOver())` koşulu altında devam eder. Her döngüde kalan hak gösterilir ve tahmin alınır.
5. **Değerlendirme:** Kullanıcının tahmini önce aralık dışı/sayısal olmayan girişlere karşı kontrol edilir. Geçerliyse, `game.evaluateGuess` çağrılır; bu metot hem hakları düşürür hem de sonucu (`HIGH/LOW/CORRECT`) döndürür.
6. **Oyun Sonu:** Oyun kazanılırsa (`won == true`) veya haklar biterse (`kalanTahmin <= 0`), döngü sonlanır ve `displayGameSummary` ile sonuç özetlenir.

2.3. Karşılaşılan Sorunlar ve Çözümleri

Sorun	Açıklama	Çözüm ve Gerekçe
Kaynak Sızıntısı (Resource Leak)	`App.java`'da `main` ve `configureGame` içinde ayrı `Scanner` nesnelerinin oluşturulması.	**`try-with-resources` Kullanımı:** `Scanner` tek bir yerde (`main` metodunda) oluşturuldu, `configureGame` metoduna parametre olarak iletildi ve `try` bloğundan çıkıldığında otomatik olarak kapatılması sağlandı.
Metot Erişimi (Static Hatası)	`main` metodundan çağrılan yardımcı metotların (`displayStatus`, vb.) `static` olmaması.	İlgili metotlara `private static` erişim belirteçleri eklenerek, statik bağlamda erişilebilirlikleri sağlandı.
Girdi Doğrulama Eksikliği	Aralık seçiminde (1/2), hatalı girdi kabul ediliyordu.	`configureGame` metoduna, kullanıcıyı sadece '1' veya '2' girmeye zorlayan bir **`while` doğrulama döngüsü** eklendi.

3. Kaynak Kodlar

3.1. Çalıştırma Adımları

1. Aşağıdaki listede yer alan tüm beş (`.java`) dosyasını aynı klasöre kaydediniz.
2. Komut satırını/terminali bu klasöre yönlendiriniz.
3. Tüm sınıfları derleyiniz.

4. Uygulamayı başlatınız.

3.2. Kod Listesi (Yorumlu)

Aşağıda, istenen tam ve güncel kaynak kodlar, her sınıfın işlevini belirten yeterli yorum satırlarıyla birlikte sunulmuştur.

Attempt.java

```
//  Tahmin detaylarının tutulduğu kodlar.

public class Attempt {

    public enum GuessResult { // Tahmin sonucunun değerini belirtmek için
kullanılacak olan enum.
        HIGH,
        LOW,
        CORRECT
    }

    private final int guessValue;
    private final GuessResult result;
    private final int attemptNo;

    // Tahmin değerini, değerin doğruluğunu ve tahmin sayısını atayan
constructor.
    public Attempt(int guessValue, GuessResult result, int attemptNo) {
        this.guessValue = guessValue;
        this.result = result;
        this.attemptNo = attemptNo;
    }

    // Değerlerin dışarıdan erişilmesini sağlayan kodlar.
    public int getGuessValue() {
        return guessValue;
    }

    public GuessResult getResult() {
        return result;
    }

    public int getAttemptNo() {
        return attemptNo;
    }
}
```

GameConfig.java

```
  // Oyunun sınırlandırıcılarının(tahmin edilecek max/min değer ve deneme hakkı)
olduğu kodlar.

public class GameConfig {    // Sınırlayıcıların tanımlanması.
    private final int min;
    private final int max;
    private final int maxAttempts;

    // Sınırlayıcıların atanmasının yapıldığı constructor.
    public GameConfig(int min, int max, int maxAttempts) {
        this.min = min;
        this.max = max;
        this.maxAttempts = maxAttempts;
    }

    // Sınırlayıcı değerlerin dışarıdan erişilmesini sağlayan kodlar.
    public int getMin() {
        return min;
    }

    public int getMax() {
        return max;
    }

    public int getMaxAttempts() {
        return maxAttempts;
    }
}
```

RandomNumberGenerator.java

```
 // Tahmin edilmeye çalışılan rastgele sayının (aralık dahilinde) üretildiği
kodlar.

import java.util.Random;

// Rastgele üretilecek sayının aralıklarının tanımlanması.
public class RandomNumberGenerator {
    private final int min;
    private final int max;
    private final Random random; // Javaya ait rastgele sayı üretici nesne.

    // Rastgele üretilecek sayının değerlerini atayan constructor.
```

```java
    public RandomNumberGenerator(int min, int max) {
        this.min = min;
        this.max = max;
        this.random = new Random();
    }

    // Rastgele sayının üretilmesi.
    public int generate() {
        return random.nextInt(max - min + 1) + min;
    }
}
```

NumberGuessGame.java

```java
 // Oyunun yönetildiği kodlar.


 // Yönetici değişkenlerin tanımlanması.
public class NumberGuessGame {
    private final GameConfig config;
    private final int sayi;
    private int kalanTahmin;
    private int yapilanTahmin;
    private boolean finished;
    private boolean won;

    // Yönetici değişkenleri atayan constructor.
    public NumberGuessGame(GameConfig config) {
        this.config = config;
        this.sayi = new RandomNumberGenerator(config.getMin(),
config.getMax()).generate();
        this.kalanTahmin = config.getMaxAttempts();
        this.yapilanTahmin = 0;
        this.finished = false;
        this.won = false;
    }

    public Attempt.GuessResult evaluateGuess(int guess) {
        if (finished) { // Oyun bittiyse işlem yapmaz.
            return null;
        }

        kalanTahmin--;
        yapilanTahmin++;
```

```java
        // Tahmine göre yönlendirilmenin yapılması.
        Attempt.GuessResult result;
        if (guess == sayi) {
            result = Attempt.GuessResult.CORRECT;
            this.won = true;
            this.finished = true;
        } else if (guess > sayi) {
            result = Attempt.GuessResult.HIGH;
        } else {
            result = Attempt.GuessResult.LOW;
        }

        if (kalanTahmin <= 0 && !won) {
            this.finished = true;
        }

        return result;
    }

    // Yönetici kodların dışarıdan erişilmesi.
    public boolean isOver() {
        return finished;
    }

    public boolean isWon() {
        return won;
    }

    public int getNumber() {
        return sayi;
    }

    public int getKalanTahmin() {
        return kalanTahmin;
    }

    public int getYapilanTahmin() {
        return yapilanTahmin;
    }

    public GameConfig getConfig() {
        return config;
    }

}
```

App.java

```
// Oyunun çalıştırılacağı ve oyuncunun yönlendirildirilmesi için çıktı verecek
olan kodlar.

import java.util.Scanner;

// Main metod
public class App {

    public static void main(String[] args) {
        System.out.println("=== NTP Sayı Tahmin Oyunu ===");

        // Kaynak sızıntısı için önlem.
        try (Scanner scanner = new Scanner(System.in)) {

            // Oyun yapılandırmasını alma ve Scannerın iletimi.
            GameConfig config = configureGame(scanner);
            NumberGuessGame game = new NumberGuessGame(config);

            System.out.printf("\nOyun Başladı! %d ile %d arasındaki sayıyı tahmin
et. Deneme hakkı: %d\n",
                              config.getMin(), config.getMax(),
config.getMaxAttempts());

            // Ana oyun döngüsü.
            while (!game.isOver()) {
                displayStatus(game);

                System.out.print("Tahmininiz: ");
                String input = scanner.nextLine().trim();

                if (input.isEmpty()) {
                    continue;
                }

                try {
                    int guess = Integer.parseInt(input);

                    // Aralık dışı değer kontrolü.
                    if (guess < config.getMin() || guess > config.getMax()) {
                        System.out.printf("Aralık dışı değer! Lütfen %d ile %d
arasında bir tam sayı girin.\n", config.getMin(), config.getMax());
                        continue;
                    }
```

```
                    Attempt.GuessResult result = game.evaluateGuess(guess);

                    if (game.isOver()) {
                        break;
                    }
                    // Oyuncuya geri bildirim.
                    switch (result) {
                        case HIGH:
                            System.out.println("Çok Yüksek");
                            break;
                        case LOW:
                            System.out.println("Çok Düşük");
                            break;
                        default:
                            break;
                    }
                    // Sayısal olmayan giriş kontrolü.
                } catch (NumberFormatException e) {
                    System.out.println("Geçersiz giriş. Lütfen bir tam sayı
girin.");
                    continue;
                }
            }

            displayGameSummary(game); // Oyun sonu özet mesajı.
        }
    }

    // Aralık ve deneme hakkının kullanıcıdan alınması.
    private static GameConfig configureGame(Scanner scanner) {
        int min = 1, max = 100, maxAttempts = 7;


        // Aralık seçiminin doğrulanması.

        boolean girdi = false;

        while (!girdi) {         // Girdi kontrolü için döngü.
            System.out.print("Aralık ? (1) 1-100 (Varsayılan deneme: 7) (2) 1-
1000 (Varsayılan deneme: 10): ");
            String rangeChoice = scanner.nextLine().trim();

            if (rangeChoice.equals("1")) {
                max = 100;
```

```java
                maxAttempts = 7;
                girdi = true;
            } else if (rangeChoice.equals("2")) {
                max = 1000;
                maxAttempts = 10;
                girdi = true;
            } else {
                // 1 veya 2 den farklı bir sayı girilirse uyarı veren ve kodlar.
                System.out.println("Hata: Lütfen sadece '1' veya '2' giriniz.");
            }
        }

        // Max deneme hakkı seçimi.
        System.out.printf("Maksimum deneme hakkı (Varsayılan %d)? : ",
maxAttempts);
        try {
            String attemptInput = scanner.nextLine().trim();
            // Boş bırakılmasına karşı önlem.
            if (!attemptInput.isEmpty()) {
                maxAttempts = Integer.parseInt(attemptInput);
            }
        } catch (NumberFormatException ignored) { }    // Varsayılan değerin
kullanılması.

        return new GameConfig(min, max, maxAttempts);
    }

    // O anki kalan deneme hakkının gösterilmesi.
    private static void displayStatus(NumberGuessGame game) {
        System.out.printf("\nKalan Deneme: %d\n", game.getKalanTahmin());
    }

    // Oyun sonu durum çıktıları.
    private static void displayGameSummary(NumberGuessGame game) {
        System.out.println("\n=== Oyun Bitti ===");

        if (game.isWon()) {
            System.out.printf("Tebrikler doğru bildiniz! %d. denemede oyunu
kazandınız.\n", game.getYapilanTahmin());
        } else {
            System.out.printf("Deneme hakkınız bitti! Doğru sayı: %d. Yine
bekleriz.\n", game.getNumber());
        }
    }
}
```

4. ÇIKTILAR / SONUÇ VE DEĞERLENDİRME

4.1. Test Senaryoları

Aşağıdaki tablo, uygulamanın farklı senaryolarda doğru çalışıp çalışmadığını göstermek için kullanılan testleri özetlemektedir.

Senaryo No	Test Edilen Durum	Beklenen Sonuç	Gerçekleşen Sonuç
T1 (Başarılı)	Sayıyı deneme hakkı içinde bulma.	"Tebrikler!" mesajı ve oyunun bitmesi.	Başarılı
T2 (Başarısız)	Tüm deneme haklarını bitirme.	"Deneme hakkınız bitti!" mesajı ve gizli sayının gösterilmesi.	Başarılı
T3 (Girdi Doğrulama)	Aralık seçiminde '3' girmek.	"Hata: Lütfen sadece '1' veya '2' giriniz." mesajı ve tekrar sorma.	Başarılı
T4 (Girdi Hata Yönetimi)	Tahmin olarak 'abc' veya '5000' girmek (1-100 aralığında).	Hak kaybetmeden uyarı (Geçersiz giriş/Aralık dışı) verilmesi.	Başarılı

4.2. Oyun Oturumu Örnekleri (Çıktılar)

Aşağıda, farklı senaryoları kapsayan en az 3 tam oyun oturumunun konsol çıktısı yer almaktadır.

Oturum 1: Başarılı Senaryo (1-100 Aralık) Doğru Sayı: 42

```
=== NTP Sayı Tahmin Oyunu ===
Aralık ? (1) 1-100 (Varsayılan deneme: 7) (2) 1-1000 (Varsayılan deneme: 10): 1
Maksimum deneme hakkı (Varsayılan 7)? :

Oyun Başladı! 1 ile 100 arasındaki sayıyı tahmin et. Deneme hakkı: 7

Kalan Deneme: 7
Tahmininiz: 50
Çok Yüksek

Kalan Deneme: 6
Tahmininiz: 25
Çok Düşük

Kalan Deneme: 5
Tahmininiz: 40
Çok Düşük

Kalan Deneme: 4
Tahmininiz: 45
Çok Yüksek
```

```
Kalan Deneme: 3
Tahmininiz: 42
=== Oyun Bitti ===
Tebrikler doğru bildiniz! 5. denemede oyunu kazandınız.
```

Oturum 2: Başarısız Senaryo (Hak Tükenmesi) Doğru Sayı: 81 (Max Deneme: 3 olarak ayarlandı)

```
=== NTP Sayı Tahmin Oyunu ===
Aralık ? (1) 1-100 (Varsayılan deneme: 7) (2) 1-1000 (Varsayılan deneme: 10): 1
Maksimum deneme hakkı (Varsayılan 7)? : 3

Oyun Başladı! 1 ile 100 arasındaki sayıyı tahmin et. Deneme hakkı: 3

Kalan Deneme: 3
Tahmininiz: 50
Çok Düşük

Kalan Deneme: 2
Tahmininiz: 90
Çok Yüksek

Kalan Deneme: 1
Tahmininiz: 70
Çok Düşük
=== Oyun Bitti ===
Deneme hakkınız bitti! Doğru sayı: 81. Yine bekleriz.
```

Oturum 3: Giriş Doğrulama Testi (1-1000 Aralık)

```
=== NTP Sayı Tahmin Oyunu ===
Aralık ? (1) 1-100 (Varsayılan deneme: 7) (2) 1-1000 (Varsayılan deneme: 10): 3
Hata: Lütfen sadece '1' veya '2' giriniz.
Aralık? (1) 1-100 (Varsayılan: 7 deneme) (2) 1-1000 (Varsayılan: 10 deneme): 2
Maksimum deneme hakkı (Varsayılan 10)? :

Oyun Başladı! 1 ile 1000 arasındaki sayıyı tahmin et. Deneme hakkı: 10

Kalan Deneme: 10
Tahmininiz: 500
Çok Düşük

Kalan Deneme: 9
Tahmininiz: 1500
Aralık dışı değer! Lütfen 1 ile 1000 arasında bir tam sayı girin.

Kalan Deneme: 9
Tahmininiz: abc
Geçersiz giriş. Lütfen bir tam sayı girin.

Kalan Deneme: 9
Tahmininiz: 750
Çok Yüksek
... (Oyuna devam eder)
```